LEVEL 2 HACKATHON

FUNDAMENTALS OF WEB DEVELOPMENT

LIBRARY MANAGEMENT SYSTEM

Project Report with Statement, Source Code, Input & Output Screenshots

Student Details	
Student Name	V.Vishnu
Register Number	20925U18182
Department	Computer Science
ID	Not provided
College	Shanmuga Industries Arts and Science College
Date of Submission	28-10-2026
GitHub Link	To be added after uploading project

Submitted as a web development project demonstrating the use of HTML, CSS and JavaScript.

1. Problem Statement

Traditional library record keeping can involve paper registers, slow searches, and difficulty tracking which books are available or issued. A simple digital interface can make common library tasks easier to understand and manage. The Library Management System project demonstrates a browser-based interface for viewing book information and recording a book issue using a structured form.

2. Objectives

• Present a clear dashboard for library activity.
• Display summary cards for total, issued and available books.
• Collect student and book details through an issue form.
• Validate required inputs before submitting.
• Show a confirmation message and issue summary.
• Build a responsive, readable interface using HTML, CSS and JavaScript.

Expected Outcome

A working front-end prototype that demonstrates the flow from entering book-issue details to viewing a confirmation screen. This sample does not use a database; records are for demonstration within the browser session.

3. Architecture

HTML defines the page structure and form fields. CSS controls layout, colours, spacing and responsive behaviour. JavaScript validates the form, handles user actions and updates the output message. The browser executes the front-end code.

Application Flow

Start → Open Dashboard → Select “Issue a Book” → Enter Student & Book Details → Validate Inputs → If invalid, show error and return to form → If valid, show issue confirmation → End.

4. Practical Implementation

The project is organized into three files: **index.html** (content and form), **style.css** (visual design) and **script.js** (validation and output). The user enters a name, register number, book ID, book title and issue date. JavaScript checks that required fields are completed and then displays a confirmation summary.

4.1 Features Implemented

1. Dashboard cards — summarize total books, issued books and available books.
2. Issue Book form — accepts student and book details.
3. Input validation — prevents empty required fields.
4. Success confirmation — shows the submitted details and status.
5. Responsive styling — helps the interface fit mobile and desktop screens.
6. Navigation buttons — provide a simple user journey between dashboard and form.

4.2 Source Code

HTML — creates the dashboard and issue form.

HTML Code

```
<!doctype html> <html lang="en"> <head> <meta charset="UTF-8"> <meta name="viewport"
content="width=device-width, initial-scale=1"> <title>Library Management System</title> <link
rel="stylesheet" href="style.css"> </head> <body> <header><h1>Library Management System</h1><p>Digital
Library Portal</p></header> <main> <section id="dashboard"> <h2>Welcome to the Digital Library</h2>
<div class="cards"> <article><h3>Total Books</h3><p>250</p></article> <article><h3>Books
Issued</h3><p>38</p></article> <article><h3>Available</h3><p>212</p></article> </div> <button
id="showForm">Issue a Book</button> </section> <section id="issueSection"> <h2>Issue a Book</h2> <form
id="issueForm"> <label>Student Name<input id="student" required></label> <label>Register Number<input
id="regno" required></label> <label>Book ID<input id="bookId" required></label> <label>Book Title<input
id="bookTitle" required></label> <label>Issue Date<input id="issueDate" type="date" required></label>
<button type="submit">Submit Issue</button> </form> <div id="result" aria-live="polite"></div>
</section> </main> <script src="script.js"></script> </body> </html>
```

CSS — applies a clean layout and mobile-friendly styling.

```
* { box-sizing: border-box; } body { margin: 0; font-family: Arial, sans-serif; background: #f4f7fb;
color: #203047; } header { background: #142c4f; color: white; padding: 22px 5%; } header h1 { margin: 0
0 6px; } main { max-width: 1000px; margin: 28px auto; padding: 0 18px; } section { background: white;
padding: 24px; margin-bottom: 22px; border-radius: 14px; } .cards { display: flex; gap: 14px;
flex-wrap: wrap; } article { flex: 1; min-width: 160px; padding: 16px; background: #e7f1ff;
border-radius: 10px; } article p { font-size: 28px; font-weight: bold; margin: 8px 0; } label {
display: block; margin: 12px 0; font-weight: bold; } input { display: block; width: 100%; padding:
11px; margin-top: 6px; border: 1px solid #b9c8db; border-radius: 7px; } button { border: 0;
border-radius: 8px; padding: 12px 18px; background: #1769aa; color: white; font-weight: bold; cursor:
pointer; } #result { margin-top: 18px; line-height: 1.7; } @media (max-width: 600px) { section {
padding: 16px; } header h1 { font-size: 22px; } }
```

JavaScript — validates the input and displays the result.

```
const form = document.getElementById("issueForm"); const result = document.getElementById("result");
document.getElementById("showForm").addEventListener("click", () => {
document.getElementById("issueSection").scrollIntoView({ behavior: "smooth" }); });
form.addEventListener("submit", function (event) { event.preventDefault(); const student =
document.getElementById("student").value.trim(); const regno =
document.getElementById("regno").value.trim(); const bookId =
document.getElementById("bookId").value.trim(); const title =
document.getElementById("bookTitle").value.trim(); const date =
document.getElementById("issueDate").value; if (!student || !regno || !bookId || !title || !date) {
result.textContent = "Please complete every field."; return; } result.innerHTML = `<h3>Book issued
successfully!</h3> <p><b>Student:</b> ${student}<br> <b>Register No.:</b> ${regno}<br> <b>Book:</b>
${bookId} — ${title}<br> <b>Issue Date:</b> ${date}<br><b>Status:</b> Issued</p>`; });
```

4.3 Project Execution

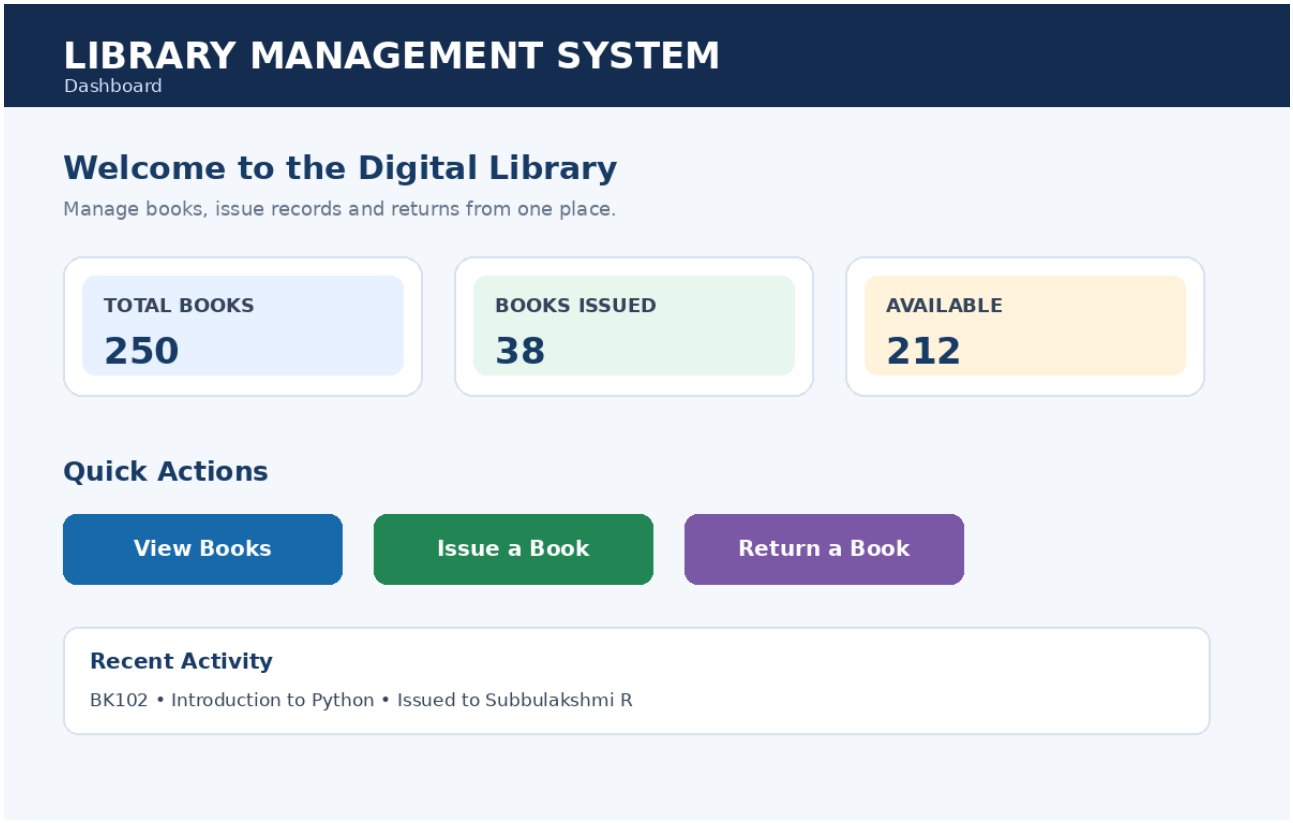
1. Create a folder named LibraryManagementSystem.
2. Save the three files as index.html, style.css and script.js in the same folder.
3. Open index.html in a browser on a laptop, or use a mobile HTML editor / online editor that supports HTML, CSS and JavaScript.
4. Click “Issue a Book”, complete all fields and select “Submit Issue”.
5. Check that the confirmation displays the entered student and book details.

Note: The dashboard numbers and issue records are sample demonstration data. For a production system, connect a backend and database and add login, book inventory updates, and secure record storage.

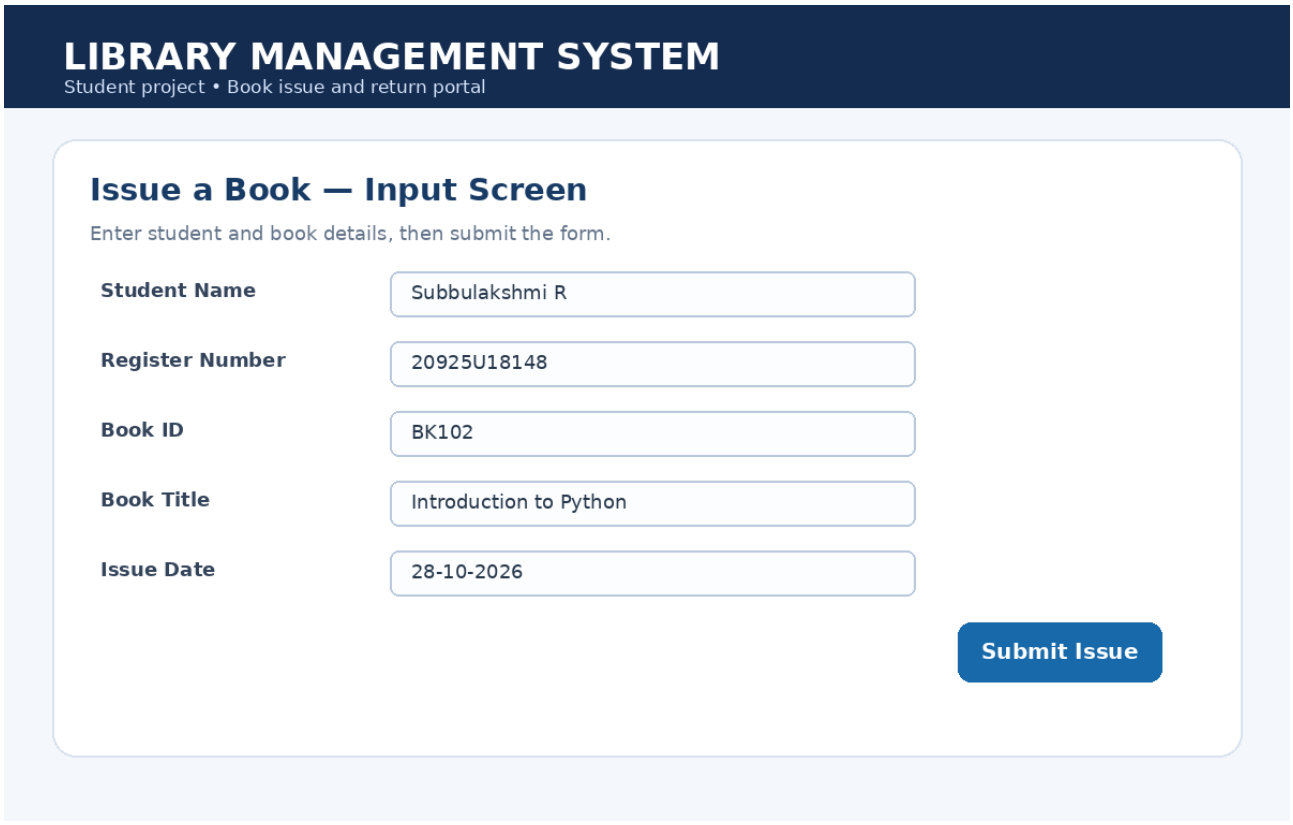
5. Output Screenshots

The following mock screenshots illustrate the expected input and output flow for the prototype. They are sample report visuals, not captures from a live deployed application.

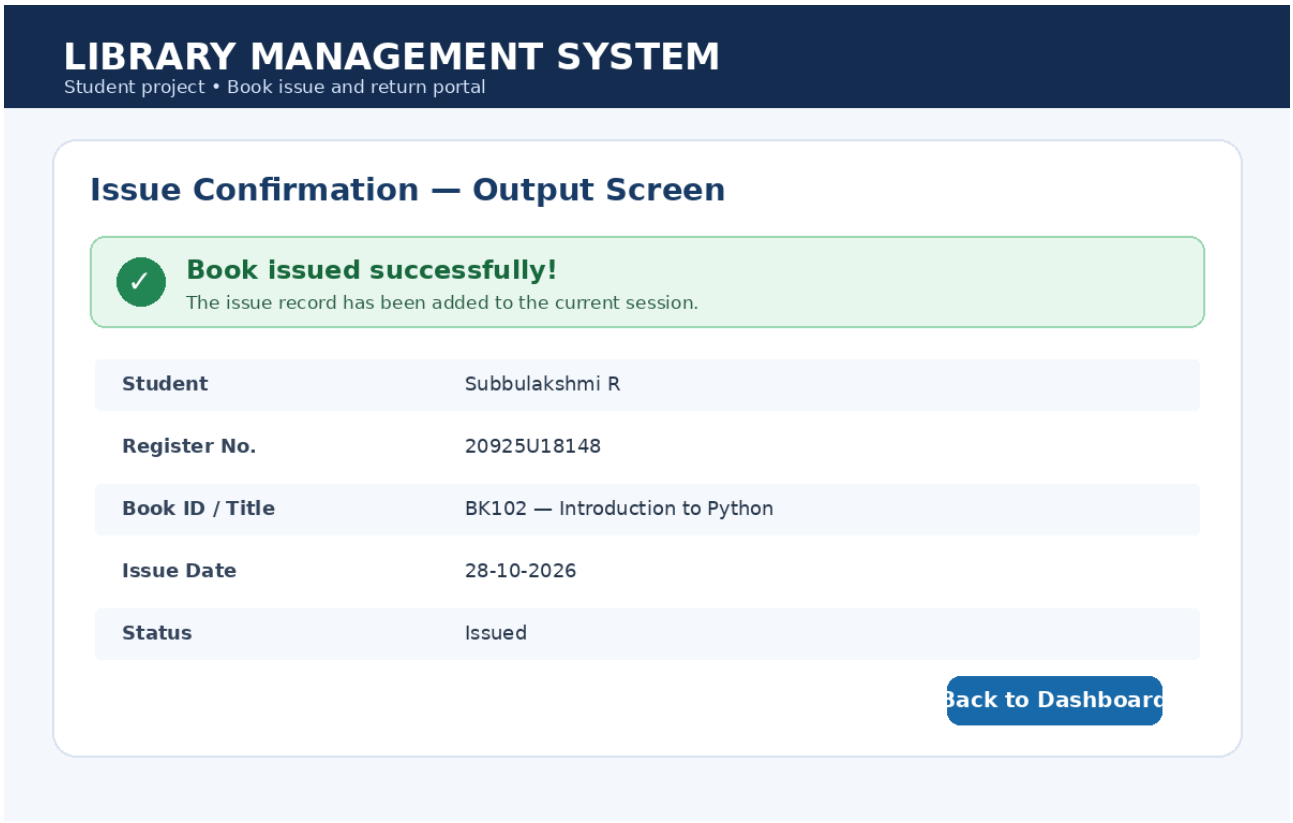
Dashboard Output



Input Screen — student and book details


LIBRARY MANAGEMENT SYSTEM
Student project • Book issue and return portal
Issue a Book — Input Screen
Enter student and book details, then submit the form.
Student Name
Subbulakshmi R
Register Number
20925U18148
Book ID
BK102
Book Title
Introduction to Python
Issue Date
28-10-2026
Submit Issue

Output Screen — issue confirmation



6. Technical Explanation

The project demonstrates semantic HTML elements, form controls, CSS flexbox and media queries, DOM selection, click and submit event listeners, input trimming, conditional validation, and dynamic result rendering. The submit event is prevented from reloading the page so that the confirmation can be shown immediately.

7. Conclusion

The Library Management System front-end prototype demonstrates how web technologies can organize a common library workflow. It provides a dashboard, an issue form, basic validation and a confirmation result. The project also gives practical experience in connecting HTML, CSS and JavaScript.

8. Future Enhancements

- Add a database to permanently store books and transactions.
- Add administrator and student login with role-based access.
- Implement search, filter, issue, return and due-date tracking.
- Automatically update availability when a book is issued or returned.
- Add overdue reminders, report export and stronger security.
- Deploy the application and include its GitHub repository link.